Module 13: Development Tools and Frameworks

Complete Development Environment

Development Stack Overview

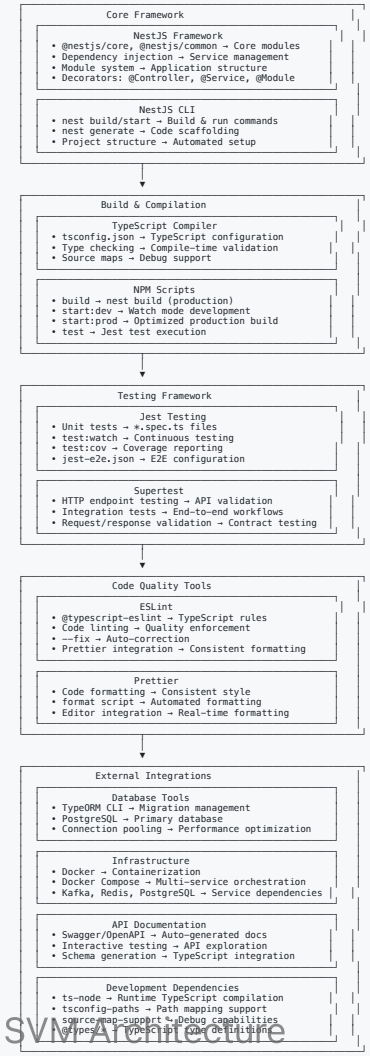
Core Technologies

- **NestJS**: Node.js framework for scalable server-side applications
- **TypeScript**: Typed JavaScript for better development experience
- **PostgreSQL**: Advanced open-source relational database
- **Redis**: In-memory data structure store for caching
- **Kafka**: Distributed event streaming platform
- **Docker**: Containerization for consistent environments

Development Principles

- **Type Safety**: Full TypeScript integration
- **Test-Driven Development**: Comprehensive testing framework
- **Code Quality**: Automated linting and formatting

Architecture Overview



NestJS Framework

Core Architecture

```
// Main application module
@Module({
  imports: [
    ConfigModule.forRoot(),
    TypeOrmModule.forRoot(databaseConfig),
    AuthModule,
    AccountsModule,
    TransactionsModule
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}

// Controller with dependency injection
@Controller('accounts')
export class AccountsController {
  constructor(private readonly accountsService: AccountsService) {}

  @Get()
  async findAll(): Promise<Account[]> {
    return this.accountsService.findAll();
  }

  @Post()
  async create(@Body() createAccountDto: CreateAccountDto): Promise<Account> {
    return this.accountsService.create(createAccountDto);
  }
}

// Service with business logic
@Injectable()
export class AccountsService {
  constructor(
    @InjectRepository(Account)
    private accountsRepository: Repository<Account>,
  ) {}

  async findAll(): Promise<Account[]> {
    return this.accountsRepository.find();
  }

  async create(createAccountDto: CreateAccountDto): Promise<Account> {
```

TypeScript Configuration

tsconfig.json

```
{
  "compilerOptions": {
    "module": "commonjs",
    "declaration": true,
    "removeComments": true,
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true,
    "allowSyntheticDefaultImports": true,
    "target": "ES2020",
    "sourceMap": true,
    "outDir": "./dist",
    "baseUrl": "./",
    "incremental": true,
    "skipLibCheck": true,
    "strictNullChecks": false,
    "noImplicitAny": false,
    "strictBindCallApply": false,
    "forceConsistentCasingInFileNames": false,
    "noFallthroughCasesInSwitch": false,
    "paths": {
      "@/*": ["src/*"],
      "@/common/*": ["src/common/*"],
      "@/modules/*": ["src/modules/*"]
    }
  },
  "include": ["src/**/*"],
```

Testing Framework

Jest Configuration

```
// jest.config.js
module.exports = {
  moduleFileExtensions: ['js', 'json', 'ts'],
  rootDir: 'src',
  testRegex: '.*\\.spec\\.ts$',
  transform: {
    '^.+\\.(t|j)s$': 'ts-jest',
  },
  collectCoverageFrom: [
    '**/*.(t|j)s',
    '!**/*.module.ts',
    '!**/main.ts',
  ],
  coverageDirectory: '../coverage',
  testEnvironment: 'node',
  moduleNameMapping: {
    '@/(.*)$': '<rootDir>/$1',
  }
```

Code Quality Tools

ESLint Configuration

```
// .eslintrc.js
module.exports = {
  parser: '@typescript-eslint/parser',
  parserOptions: {
    project: 'tsconfig.json',
    tsconfigRootDir: __dirname,
    sourceType: 'module',
  },
  plugins: ['@typescript-eslint/eslint-plugin'],
  extends: [
    'eslint:recommended',
    '@typescript-eslint/recommended',
    'prettier',
  ],
  root: true,
  env: {
    node: true,
    jest: true,
  },
  ignorePatterns: ['.eslintrc.js'],
  rules: {
    '@typescript-eslint/interface-name-prefix': 'off',
    '@typescript-eslint/explicit-function-return-type': 'off',
    '@typescript-eslint/explicit-module-boundary-types': 'off',
    '@typescript-eslint/no-explicit-any': 'off',
```

Database Tools

TypeORM Configuration

```
// data-source.ts
export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT) || 5432,
  username: process.env.DB_USERNAME || 'postgres',
  password: process.env.DB_PASSWORD || 'password',
  database: process.env.DB_NAME || 'solana_svm',
  synchronize: false, // Use migrations in production
  logging: process.env.NODE_ENV === 'development',
  entities: ['dist/**/*.entity{.ts,.js}'],
  migrations: ['dist/database/migrations/*{.ts,.js}'],
  subscribers: ['dist/database/subscribers/*{.ts,.js}'],
});
```

Migration Management

Docker & Infrastructure

Multi-Stage Dockerfile

```
# Development stage
FROM node:18-alpine AS development
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine AS production
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY --from=development /app/dist ./dist
COPY --from=development /app/migrations ./migrations
USER node
EXPOSE 3000
```

API Documentation

Swagger Integration

```
// main.ts
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  const config = new DocumentBuilder()
    .setTitle('Solana SVM Study API')
    .setDescription('API for Solana SVM blockchain operations')
    .setVersion('1.0')
    .addTag('accounts', 'Account management endpoints')
    .addTag('transactions', 'Transaction operations')
    .addTag('signing', 'Cryptographic signing operations')
    .addBearerAuth()
    .addApiKey({ type: 'apiKey', name: 'X-API-Key', in: 'header' }, 'api-key')
    .build();

  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);
  await app.listen(3000);
```

Development Workflow

Development Scripts

```
{
  "scripts": {
    "build": "nest build",
    "format": "prettier --write \"src/**/*.ts\" \"test/**/*.ts\"",
    "start": "nest start",
    "start:dev": "nest start --watch",
    "start:debug": "nest start --debug --watch",
    "start:prod": "node dist/main",
    "lint": "eslint \"{src,apps,libs,test}/**/*.ts\" --fix",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:cov": "jest --coverage",
    "test:debug": "node --inspect-brk -r tsconfig-paths/register -r ts-node/register node_modules/.bin/jest --runInBand",
    "test:e2e": "jest --config ./test/jest-e2e.json"
  }
}
```

Development Environment Setup

```
# Install dependencies
npm install

# Start development database
```

Key Takeaways

Development Environment Benefits

- **Full-Stack TypeScript**: Type safety from database to API
- **Comprehensive Testing**: Unit and integration test coverage
- **Code Quality Automation**: Linting and formatting enforcement
- **Containerized Development**: Consistent environments across teams
- **API Documentation**: Auto-generated, interactive documentation

Tool Integration Benefits

- **NestJS Framework**: Scalable, maintainable application structure
- **TypeORM**: Type-safe database operations with migrations
- **Jest + Supertest**: Complete testing framework for APIs
- **Docker Compose**: Multi-service development environment